NetSec 3D: Final Technical Report

Project: Intelligent network security with IDS, IPS, Zero Trust, SIEM and EDR on a simulated smart campus, shown in 3D. **Branch:** `feature/campus-c1-design` (nothing pushed, nothing deployed).

> **Read this first.** All traffic in this project is **synthetic and deterministic**. No real packet is sent, captured or blocked, the endpoint CLI is not a real shell, and every number below comes from the simulation engine or from the test runs listed in section 18. Status labels used throughout: **Implemented and tested**, **Implemented, partly tested**, **Not done / planned**.

---

1. Executive summary

NetSec 3D is a teaching and demonstration application. A Python backend runs a deterministic discrete-event simulation of a campus network (30 devices plus an abstract Internet segment, 32 links, 4 VLANs, 7 IP subnets) and returns a time-stamped trace. A React and Three.js front end replays that trace as a 3D campus with moving packets, device panels, a SIEM console and an endpoint CLI. The backend decides what happens to every packet; the front end only draws it.

The project demonstrates five security capabilities (IDS, IPS, Zero Trust, SIEM, EDR) through nine presentation scenarios (P1 to P9) plus the original seven scenarios (S1 to S7). The last full verification passed: 1210 backend tests, 93 of 93 scenario checks across 12 runs, 363 frontend tests, a production build, 70 of 70 browser interaction checks, 39 of 39 browser demonstration checks, the split-origin checks (20 of 20 and 34 of 34) and the offline recovery check (8 of 8).

Known weakness: on the integrated Intel UHD GPU used for testing, High quality runs at 26 to 33 fps (the idle overview, at 26 fps, is below the 30 fps target); Low quality runs at 60 fps (section 19).

2. Objectives

1. Show how a firewall, ACLs, IDS, IPS, Zero Trust, SIEM and EDR cooperate, and which component makes each decision.
2. Keep IDS passive and IPS inline, and never blur the two.
3. Make every outcome reproducible, so a presenter can repeat any scenario and get the same result.
4. Present it as a recognisable 3D campus that stays responsive on ordinary laptops.

3. Scope and what is out of scope

In scope: the simulation, the nine presentation scenarios, the 3D campus, the panels, the SIEM console, the EDR isolate and restore actions, the endpoint CLI and a read-only device CLI. Out of scope (and not claimed):

real traffic, real attack tooling, machine learning detection, a database, user accounts, real device configuration, and any deployment of the work.

4. System architecture

- **Backend:** Python, FastAPI, a deterministic discrete-event engine (`backend/netsec/sim/`), security modules (`backend/netsec/security/`), CLI (`backend/netsec/cli/`), API routes (`backend/netsec/api/`).
- **Front end:** React 19, Vite, React Three Fiber, drei, postprocessing, Zustand. A trace player applies events at the playhead.
- **Contract:** `contracts/events.json` (version 1.1.0) defines the trace events; `contracts/topology_original.json` pins the original topology.
- **Authority:** the backend produces the whole trace. Operator actions (isolate, restore) are control events at the playhead, after which the engine re-simulates; everything before the playhead stays identical.
- **Production mode:** read-only. State-changing actions return HTTP 403 and runs do not write server state. Status: **Implemented and tested.**

5. Network topology

- 31 nodes (30 devices and the abstract `INET` segment), 32 links, 4 VLANs (10 Staff, 20 Admin, 30 Servers, 99 Security), 7 IP subnets.
- The original 17 nodes and 18 links, their ids and addresses are unchanged (a test pins them to `contracts/topology_original.json`). The extension adds 14 devices and 14 links: more staff and admin PCs, extra servers, and the security components (SIEM-1, EDR-MGMT-1, ZT-POLICY-1).
- Path of a staff request to the application server: PC, ACC-SW2, CORE-SW1 (ACL and Zero Trust enforcement point), IPS-1 (transparent, inline), SRV-SW3, APP-SRV. Status: **Implemented and tested.** See `docs/NETWORK_TOPOLOGY.md`.

6. IDS (passive detection)

IDS-1 receives SPAN copies of traffic entering CORE-SW1 (not VLAN 99). It raises alerts and syslog only. It never forwards, drops, blocks or rate-limits, and it never recolours the original packet. Detectors: D1 port scan, D2 brute force, D3 flood, D4 restricted service, D5 forbidden zone pair, D6 invalid TCP flags, with thresholds in the shipped rules. A test asserts that IDS-1 never emits a drop, block or IPS action event. Status: **Implemented and tested.**

7. IPS (inline prevention)

IPS-1 is a transparent bridge in VLAN 30. Order for each packet: block table, detectors on every arriving packet, policy for the current mode, enforcement (drop, reject, block source, rate-limit), forward. **Monitor** records "would block" and drops nothing; **Enforce** applies the action. Mode is chosen per run. Status: **Implemented and tested** (scenarios S2, S4 to S7 exercise both modes).

8. Firewall and ACL (static policy)

FW1 (stateful, with connection tracking) and the CORE-SW1 ACL `STAFF-IN` enforce static policy regardless of IPS mode. For example STAFF-IN rule 10 denies staff to DB-SRV tcp/3306, rule 20 denies staff to the Security

VLAN, rule 100 permits the rest; FW1 OUTSIDE-IN rule 30 publishes HTTP to APP-SRV and everything else is denied by the implicit rule. Status: **Implemented and tested.**

9. Zero Trust

ZT-POLICY-1 is asked once per flow by the policy enforcement point CORE-SW1, after the ACL. A decision uses endpoint trust, role and the requested resource, and records its reasons. Example (P4): the admin role is explicitly permitted tcp/443 on SIEM-1 and tcp/8443 on EDR-MGMT-1; being an administrator alone grants nothing. Status: **Implemented and tested.**

10. SIEM

SIEM-1 receives events from the components that produced them (IDS, IPS, firewall, ACL, Zero Trust, EDR), groups related events into incidents (incident ids are scoped per run) and keeps an incident history. The front end shows a SIEM console on the Threats page and security events in the player. Status: **Implemented and tested** (backend tests and browser demonstration check).

11. EDR with isolate and restore

Each endpoint has a state (healthy, compromised, isolated). EDR-MGMT-1 receives agent reports over an out-of-band channel (not simulated as packets). The operator isolates or restores an endpoint from the endpoint panel (Security tab) as a control at the playhead; the result is committed to the server state, so the engine, the terminal and the labels read one state. **What isolation does in the engine** (`backend/netsec/sim/engine.py`): 1. An isolated endpoint creates **no packet**: no envelope leaves it, nothing reaches ACC-SW2, CORE-SW1, IDS-1 or IPS-1. One `edr.send_blocked` event per flow records the stopped attempt (the SIEM-style ticker shows it). 2. A packet the endpoint sent *before* the isolation, still on a cable or inside a device when it takes effect, is discarded with reason `edr_isolated` on arrival or departure, before any policy, mirror, sensor or IPS looks at it. Events before the playhead keep their timestamps and outcomes. 3. Packets sent *to* an isolated endpoint are dropped at its agent. Other endpoints are unaffected. 4. Commands on an isolated endpoint (`ping`, `tracert`, `telnet`, `curl`) fail at once with "TRANSMIT FAILED: this endpoint is ISOLATED by EDR" and animate nothing.

Root cause of the bug that was fixed: the scenario engine kept *generating* a flow's packets for an isolated source (it created each packet and dropped it at the endpoint), and packets sent just before the isolation kept crossing switches and being inspected by IDS-1 and IPS-1 (the code even documented that they "are delivered normally"). Drops and alerts after the isolation therefore looked like new traffic from PC-2. Regression tests: `backend/tests/security/test_isolation_and_ping.py` (isolations at 4, 5 and 9 s: zero packets created after, none inspected, no alerts from non-existent traffic, history unchanged, PC-1 unaffected, recovery restores communication, same state in the CLI and the engine). No malware or operating-system process is involved: it is a state machine plus the rules above. Status: **Implemented and tested.**

12. Presentation scenarios P1 to P9

Every scenario has a name, objective, source, destination, protocol and port, a Run button, a Reset button, an animation, events, an expected outcome, an explanation and SIEM events, and can be repeated with the same

result. The table lists the design values from `backend/config/scenarios/P*.json`; the outcomes are asserted by `backend/tests/scenarios/test_presentation_scenarios.py` and checked again in the browser (section 18).

#	Name	Source to destination	Protocol / port	Duration	Expected outcome
P1	Normal communication	PC-1 to APP-SRV	TCP 443	18 s	Delivered; no alert, no IPS action, no SIEM event
P2	Unauthorized Security VLAN access	PC-1 to SIEM-1	ICMP	6 s	Dropped at CORE-SW1 by STAFF-IN:20; no reply; SIEM gets an enforcement event
P3	Database access restriction	PC-1 to DB-SRV	TCP 3306	3 s	SYN dropped at CORE-SW1 by STAFF-IN:10; never reaches IPS-1 or DB-SRV
P4	Authorized administrator access	ADMIN-PC to SIEM-1	TCP 443	13 s	Two Zero Trust allows for ADMIN-PC; PC-1's attempt is stopped by STAFF-IN:20
P5	External HTTP access	EXT-USR to APP-SRV	TCP 80	25 s	Crosses R1-EDGE and FW1 (OUTSIDE-IN:30), reply allowed by connection tracking, delivered
P6	External database access restriction	EXT-ATK to DB-SRV	TCP 3306	3 s	Dropped at FW1 (implicit deny); never enters the campus; IDS-1 cannot see it
P7	Compromised endpoint and EDR isolation	PC-2 to APP-SRV	TCP	32 s	EDR alert at 1 s; D1 scan detected by IDS-1 and IPS-1; after isolation PC-2's packets are dropped with `edr_isolated`; other endpoints keep working
P8	Endpoint recovery	PC-2 to APP-SRV	TCP 443	26 s	Pings dropped while isolated; restore at 5 s; PC-2 healthy; HTTPS works again
P9	SIEM event monitoring	PC-3 to APP-SRV	TCP 443	7 s	IDS-1 and IPS-1 each raise D6; the SIEM lists the events as one incident

Status: **Implemented and tested.**

13. Original scenarios S1 to S7

The original seven scenarios are unchanged. `test_extension_equivalence.py` shows that their events are identical to the golden traces apart from the run id and configuration hash, because the new security features are opt-in per scenario. The shipped-rules check passes 12 of 12 runs and 93 of 93 checks (S1 to S7, Monitor and Enforce).

14. Endpoint CLI

An interactive terminal in the endpoint panel supports `help`, `ipconfig`, `ping`, `tracert`, `telnet`, `curl` and `clear`. Commands are whitelisted and validated by regular expressions; nothing is executed on any machine. Each network command runs the real simulation engine, so its output (replies, time-outs, the hop where it stopped, the enforcing device and rule) comes from the backend's decisions, not from a script. **Ping is four sequential journeys.** The four echo requests (ICMP type 8) leave one after another: attempt *n+1* departs only after attempt *n* has ended (its reply, type 0, reached the source, or the request or the reply was stopped). Each attempt has its own colour for its whole journey, request and reply alike: cyan blue `#38BDF8`, purple `#A78BFA`, amber `#F59E0B`, green `#34D399`. The packet is drawn as a 3D envelope that follows the cable path of the route computed from the topology; a green tick appears when it reaches the destination and again when the reply reaches the source; a packet that is stopped shows a red cross at the enforcing device (CORE-SW1 for Staff to Security or Database, FW1 for the external user to the database or the Security network) and goes no further. **The terminal is synchronised with the packets.** The backend returns, with the output, a time for every line

(`timeline`): a `Reply from ...` line is stamped with the moment its reply reached the source, a time-out with the moment the request was stopped. The terminal prints a line only when the playhead has reached that time (`frontend/src/features/simulation/cliTimeline.js`), so pausing, scrubbing and changing the playback speed move the lines with the animation. The time shown in `time=...ms` is a synthetic figure (0.4 ms per link crossed), independent of the animation speed. **Stop / Ctrl+C** keeps the lines already printed, adds `^C` and the statistics of what really happened (marked interrupted), unloads the run and leaves no animation or counter behind. Status: **Implemented and tested** (`backend/tests/security/test_isolation_and_ping.py`, `test_campus_security.py`; `cliTimeline.test.js`; the demonstration check asserts that no reply line is printed while the first request is still travelling, that the first reply line appears only after its reply packet was delivered, that the second request is not sent earlier, and that Stop works).

15. Device CLI (read-only)

`show` commands for R1-EDGE, CORE-SW1 and FW1 produce output from the topology, the rules and the hit counters. They cannot change anything. Status: **Implemented and tested** (`backend/tests/security/test_device_cli.py`).

16. 3D campus

- **Composition:** the facilities stand close together around the core hub on one dark navy ground plate (about 80 by 60 units) with no roads, paths or bridges; only thin cyan cables join them. Each of the seven zones has a recognisable glass-and-steel building: continuous blue-tinted reflective glass on all four sides, steel-blue frames with a top band and corner posts, a defined entrance in the south wall and no roof, so the interiors stay visible. Buildings differ in height (Administration is the tallest). A glass cylinder surrounds the core switch. Each building has a large sign on posts above its back wall. There are no car parks, vehicles, vegetation or footpaths.
- **Devices** are drawn 1.8 times their original model size. Procedural models: desks with drawer units, lamps, mugs and notepads, monitors showing drawn desktop screens, towers, keyboards, mice and office chairs with armrests; rack servers with bezels, handles, vents, drive bays and status lights (different for APP-SRV, DB-SRV, FILE-SRV, EDR-MGMT-1 and ZT-POLICY-1); server racks with rack-unit marks; a core switch with two port rows on a stepped plinth; a router with two interface rows; and FW1 as a gateway built from three appliances with interface rows, vents, status lights and a shield emblem.
- **Data centre interior (decoration only, no topology device):** the five servers and SRV-SW3 stand in three racks close together; a row of network cabinets (patch panels, rack switches, cable management), a NAS storage cabinet and two backup units stand along the north wall; two cooling units stand against the east wall; perforated floor tiles, overhead cable trays with drops to the racks, spare cabinets, labelled equipment and a monitoring desk with monitor and chair fill the rest. A test checks that none of it overlaps a device, a cable or the walls.
- **Other interiors:** wall cabinets in the laboratory and the administration building, a reception desk with two screens in the visitor reception, and the operations centre's monitoring wall and operator stations.
- **Live screens:** the monitoring wall, the SOC operator desks, the reception and the data-centre monitor draw the loaded run, up to the playhead: SIEM events, packet counters (clean, flagged, blocked, alerts, incidents), endpoint status from the EDR agents, and run status (scenario, IPS mode, time, blocks, Zero Trust decisions). They are redrawn about twice a second from the same trace state the SIEM console uses; with no run loaded they say so. They show simulated data only. The small text on the wall is hard to read at the overview zoom.
- **Cables and packets:** cables are thick navy-blue tubes with a cyan ridge of light, so they read on both the pale platforms and the dark ground. While a packet is in flight, the whole cable route it will take

glows in the packet's colour and a pulse with a bright head and a fading tail travels along the cable it is on, in its direction (the return trip reverses it); a blocked packet lights its route only up to the enforcing device, where the red cross appears; the mirrored copy to IDS-1 is shown on the SPAN cable in violet, apart from the original path. This works for every scenario, not only ping (`frontend/src/scene/links/cableShader.js`, `routes.js`, `linkActivity.js`, `packets/PacketLayer.jsx`, `effects/MarkLayer.jsx`).

- Picking: devices first, then cables, then zones. A browser check clicks every one of the 31 devices, every one of the 32 cables and all 7 zones. Status: **Implemented and tested.**

17. Front end features

Scenario launcher, endpoint panel (Overview, CLI, Security tabs), device panel, packet panel, playback bar with event markers, camera presets, High/Low quality with automatic fall-back, SIEM console, Monitoring and Threats pages, and a banner that says the traffic is synthetic. UI polish done in this stage: the quality notice moved out of the scene. Other items remain minor (see section 22).

18. Testing and validation

Last full run on the final build:

Check	Result
Backend tests (pytest)	1210 passed
Scenario checks S1 to S7 (shipped rules)	12 of 12 runs, 93 of 93 checks
Frontend tests (Vitest)	30 files, 363 tests passed
Production build	succeeded
Browser interaction check (`scripts/check_interactions.mjs`)	70 of 70
Browser demonstration check (`scripts/check_demo_scenarios.mjs`)	39 of 39 (including the progressive ping output, the sequential requests and Stop)
Split-origin check (`python scripts/tasks.py split-check`)	20 of 20 checks, and the 34-step demo rehearsal 34 of 34, with the API on another origin (local only; nothing was run on Render)
Offline recovery check (`scripts/check_offline_recovery.mjs`)	8 of 8: the backend alone serves the app, "Engine offline" shows within 15 s, the loaded run is kept and scrubbable offline, the page recovers by itself, a new run works, and no request leaves 127.0.0.1

Shipped-rules trace hashes (from the verification log): S1 monitor 57cf6670ce8a, S1 enforce 0faa2b791f6e, S2 monitor 307190852d2a, S2 enforce 44dd9e4738d8, S3 monitor bf20f361e402, S3 enforce ac6bc5d49145, S4 monitor 3ad9ecd2b964, S4 enforce 3c3d85f3f55b, S5 monitor 85dc9ea334fa, S6 enforce e1b4b9c05cc5, S7 monitor fabf892d87b9, S7 enforce bc1b9cbdbeb2 (rules hash 79c0676ad9f6). They are unchanged by the visual work, which touches no backend code. The tests ran on one Windows machine; no other platform was tested.

19. Performance

Measured with `scripts/measure_matrix.mjs` (results in `docs/perf/final-matrix.json`) on an Intel UHD Graphics laptop GPU through the app's built-in browser, with the scene drawing 358 to 368 calls and about 101k to

103k triangles. No console errors were seen. Frame rates vary by a few fps between runs on this machine (earlier runs of the same build gave 24.5 to 26.5 fps for the High idle overview).

Quality	Case	Frame rate	Worst 1% frame
High	idle, overview	26.4 fps	116.7 ms
High	idle, staff close-up	33.0 fps	50 ms
High	scenario 7 playing	32.2 fps	50.4 ms
High	S4 flood peak	32.6 fps	50.2 ms
Low	idle, overview	60.1 fps	17 ms
Low	idle, staff close-up	60.1 fps	17.1 ms
Low	scenario 7 playing	60.2 fps	17.3 ms
Low	S4 flood peak	60.2 fps	17.1 ms

Assessment: the 30 fps target is met on Low and in three of four High cases, and **missed in the High idle overview (26.4 fps)** on this integrated GPU. Low quality runs at 60 fps with the same geometry, so the High cost is post-processing and fill rate, not geometry. Changes made to reduce it without removing visual detail: the outline pass exists only while a device is selected, bloom uses five mip levels and ambient occlusion fewer samples. These gave a gain within the run-to-run noise. The application lowers quality automatically when the frame rate stays low and tells the user (the notice sits at the top right of the scene). A discrete GPU was not available, so no claim is made about one.

20. Security design principles

- Backend authority: the front end never decides an outcome.
- IDS is passive; only IPS (and static FW and ACL policy, Zero Trust and EDR) enforce.
- Defence in depth: perimeter firewall, core ACL, Zero Trust, inline IPS and endpoint isolation each act at a different place.
- Production is read-only; the API validates input, limits request rates and whitelists CLI commands.

21. Determinism and reproducibility

The same scenario, seed and rules give the same trace hash. Mid-run changes leave everything before the playhead identical. This is what makes every scenario repeatable in front of an audience.

22. Limitations

Listed in `docs/LIMITATIONS.md`. Principally: simulated traffic only, simplified protocol behaviour, no machine learning, no persistence beyond a run, a High-quality idle-overview frame rate of 26 fps on the integrated GPU tested, no multi-platform test, the small text on the SOC wall being hard to read at the overview zoom, and building interiors that are less dense than the supplied reference image.

23. Risks and how they are handled

Risk	Handling
Low frame rate on the demo machine	Low quality and automatic fall-back; backup recording recommended
Operator actions on a shared server	Production is read-only; actions are available only in the local demo

Risk	Handling
Misreading synthetic results as real	The banner and every document say the traffic is synthetic

24. How to run

```
cd C:\dev\NetSec-3D
python scripts/tasks.py setup
python scripts/tasks.py demo        # builds and serves everything on one port
python scripts/tasks.py verify      # all automated checks
```

See `docs/DEVELOPMENT.md` and `docs/DEMO_SCRIPT.md`.

25. Future work (not done)

Further High-quality optimisation for integrated GPUs; measuring on other machines; more interior detail; more device CLI commands; persisted run history.

26. Conclusion

The five security capabilities, the nine presentation scenarios, both CLIs and the expanded 3D campus are implemented and pass the automated checks listed above. The open items that matter are the High-quality idle-overview frame rate on an integrated GPU (26 fps), the small text on the SOC wall at overview zoom, and interiors that are less dense than the supplied reference image. Nothing here claims real network behaviour: it is a deterministic simulation built to explain how the defences work together.